

BPCL RO NETWORK PLATFORM

CODEBASE & ARCHITECTURE REPORT

*A Technical Walkthrough of key code blocks
in the FastAPI Backend and React Frontend*

System Architecture & Core Source Code Analysis

This report delivers a deep technical breakdown of the source code powering the BPCL Retail Outlet Monitoring Platform. It covers database configurations, REST API endpoints, analytical computation logic in Python, React routing structures, dynamic inline filters, and chart rendering modules.

ORGANIZATION:

Bharat Petroleum Corporation Limited (BPCL)
RO Monitoring Platform Software Development Team

DOCUMENT RELEASE & DATE:

Developer Reference Guide v1.0 - July 2026

1. Backend Architecture & Database Models

Evaluating database schema mapping and REST API core structures

Database Connection (app/database/db.py)

The application configures a centralized SQLite engine (which can be seamlessly swapped to PostgreSQL for production). It sets up a SQLAlchemy declarative base and local session generator:

- engine = create_engine(DATABASE_URL, connect_args={'check_same_thread': False})
- SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
- Base = declarative_base()

Data Models (app/models/)

Four primary models are mapped to the relational database tables to track the BPCL retail structure:

- Site (models/site.py): Represents individual retail outlets. Contains fields for ro_id (unique string code, e.g., '110023'), ro_name, sales_area, vendor_name, location coordinates, capacity, and active status.
- Device (models/device.py): Represents hardware modules (e.g. pumps, ATGs, sensors) associated with a site. Maps device status (online, offline, maintenance), feed pressure, permeate flow, and energy stats.
- SensorReading (models/sensor_reading.py): Stores telemetry metrics. It holds onboarded_mpd, online_mpd, onboarded_tanks, online_tanks, and historical percentages. The database indexes this table by timestamp to ensure fast queries.

Backend Routing (app/main.py)

The API server initializes a FastAPI app with cross-origin resource sharing (CORS) enabled so the React dev server can request data. It mounts routers for sites, devices, sensor readings, and IoT comparison:

- app.include_router(sites.router, prefix='/sites', tags=['Sites'])
- app.include_router(iot_comparison.router, prefix='/iot-comparison', tags=['IoT Comparison'])

2. Backend Telemetry & Comparison Logic

Source code review of the core analytical calculations

Dynamic Area Grouping (app/api/iot_comparison.py)

The /iot-comparison/by-area endpoint calculates device percentages and gaps across all geographic areas. For each site, the code loads the latest sensor telemetry, aggregates MPD and Tank counts, and updates the group stats:

```
# Aggregation loop in Python:
for s in sites:
    key = s.sales_area or 'Unassigned'
    if key not in areas:
        areas[key] = { ... }
    pfx = 'iot' if s.iot_enabled is True else 'non'
    # Fetch latest reading
    r = db.query(SensorReading).filter(SensorReading.site_id == s.id).order_by(SensorReading.timestamp.desc()).first()
    if r:
        areas[key]['{pfx}_onb_mpd'] += r.onboarded_mpd or 0
        areas[key]['{pfx}_onl_mpd'] += r.online_mpd or 0
        areas[key]['{pfx}_onb_tank'] += r.onboarded_tanks or 0
        areas[key]['{pfx}_onl_tank'] += r.online_tanks or 0
```

Gap Calculations

After grouping the raw telemetry numbers, the endpoint computes percentages and comparison metrics for both groups:

- $\text{iot_mpd_pct} = (\text{iot_onl_mpd} / \text{iot_onb_mpd}) * 100$
- $\text{non_mpd_pct} = (\text{non_onl_mpd} / \text{non_onb_mpd}) * 100$
- $\text{mpd_gap} = \text{iot_mpd_pct} - \text{non_mpd_pct}$
- $\text{tank_gap} = \text{iot_tank_pct} - \text{non_tank_pct}$

The calculated data is sorted in descending order based on the total number of outlets in each area, and is sent to the frontend as a clean JSON payload.

3. Frontend Architecture & Page Shell

Technical breakdown of React routing, API services, and page layouts

React Shell & Lazy Loading (src/App.jsx)

The App.jsx file sets up the layout shell with a persistent sidebar navigation and lazy-loads the page components within a React Suspense block. Pages are rendered dynamically based on the URL path:

- `<Route path='/dashboard' element={}<Dashboard /> } />`
- `<Route path='/sites' element={}<Sites /> } />`
- `<Route path='/iot-comparison' element={}<IoTComparison /> } />`

API client (src/services/api.js)

Communication with the backend uses a standardized Axios client. Endpoints return data asynchronously:

- `export const getSites = () => axios.get(`${API_BASE}/sites/`);`
- `export const getFleetStatus = () => axios.get(`${API_BASE}/dashboard/fleet-status`);`
- `export const getIoTByArea = () => axios.get(`${API_BASE}/iot-comparison/by-area`);`

Shared Card Component (src/components/)

Uptime statistics are rendered in a shared component. It takes labels, onboarded, and online props, calculates percentages, and renders a colored progress bar indicating performance:

- `const pct = onboarded > 0 ? Math.round((online / onboarded) * 100) : 0;`
- `return (<div className='card'><div className='pct-bar' style={{ width: `${pct}%` }} /></div>)`

4. Heading Filters & Chart Styling Code

Implementing column filters and preventing label overlaps in Recharts

Dynamic Header Filters (src/pages/Sites.jsx)

To allow operators to search and filter outlets right inside the table headers, the Sites page uses React state variables coupled with a useMemo filtering hook. Dropdowns are populated dynamically from unique values in the data:

```
const uniqueAreas = useMemo(() => {  
  return Array.from(new Set(sites.map(s => s.sales_area).filter(Boolean))).sort();  
}, [sites]);
```

```
const filtered = useMemo(() => {  
  return sites.filter(s => {  
    if (selectedArea && s.sales_area !== selectedArea) return false;  
    if (selectedVendor && s.vendor_name !== selectedVendor) return false;  
    const offMpd = s.onboarded_mpd - s.online_mpd;  
    if (mpdOfflineOnly && offMpd === 0) return false;  
    return true;  
  });  
}, [sites, selectedArea, selectedVendor, mpdOfflineOnly]);
```

Overlapping Label & Legend Fixes

Long Sales Area names (e.g. 'Calcutta - 5 Retail') would overlap horizontally and collide with the chart legends. This is fixed in all charts using steeper rotations, reserved vertical spacing, and pushed legend alignments:

- ResponsiveContainer height is expanded to 320px.
- <XAxis dataKey='name' angle={-45} textAnchor='end' height={80} interval={0} />
- <Legend verticalAlign='bottom' height={36} wrapperStyle={{ paddingTop: 10 }} />

Setting height={80} on the XAxis reserves exactly 80 pixels for the tilted labels. This forces Recharts to draw the Legend below the reserved space, resolving all overlap issues.